OPERATING SYSTEMS · FALL 2026 · PROJECT 2 · TUTORIAL 1

From Linked Lists to Embedded Free Lists

Build the intuition behind `pool.c`. Start with a classic heap-allocated linked list, then see how the same pointers work when every node lives inside one contiguous byte array.

---

1. A Classic C Linked List

A standard singly-linked list in C stores each node in its own heap allocation. Every node carries a data field and a pointer to the next node:

```
typedef struct node {
    int value;
    struct node *next;
} node_t;
```

Inserting at the head

Each insertion calls `malloc` for a brand-new chunk of memory. The new node's `next` pointer is set to the current head, and the head pointer is updated:

```
node_t *insert_head(node_t *head, int val) {
    node_t *n = malloc(sizeof(node_t));
    n->value = val;
    n->next  = head;
    return n;   /* caller sets: head = insert_head(head, 42); */
}
```

Traversal

```
void print_list(node_t *head) {
    node_t *cur = head;
    while (cur != NULL) {
        printf("%d\n", cur->value);
        cur = cur->next;
    }
}
```

What it looks like in memory

Each `malloc` returns an independent address. The pointers bridge them:

```
  head
   |
   v
```

```
+-------+      +-------+      +-------+
| val=3 |      | val=7 |      | val=1 |
| next--+----> | next--+----> | next--+----> NULL
+-------+      +-------+      +-------+
 0x4A20         0x4C80         0x4E10

 Three separate malloc allocations at arbitrary addresses
```

The key property: nodes live at whatever addresses `malloc` happened to return, and the `next` pointer is the only thing that connects them.

2. What Changes When the List Is Embedded

Instead of calling `malloc` for each node, imagine making **one large allocation** up front—the *pool*. Every node (now called a *block header*) sits inside that single byte array, with a region of payload capacity immediately after it. The cache reserves this space but stores no object data in it:

```
pool->bytes
 |
 v
+--------+-----------+--------+-----------+--------+-----------+
| header | payload   | header | payload   | header | payload   |
+--------+-----------+--------+-----------+--------+-----------+
|<--- block 1 ------>|<--- block 2 ------>|<--- block 3 ------>|
|                                                              |
|<-----------------  one malloc allocation  ------------------>|
```

The `next` pointer still connects headers in order, but now every pointer falls within the same contiguous allocation. You don't ask `malloc` for each node—you *carve* nodes out of existing space.

- A **free block's** payload is unused memory, available for a future allocation.
- An **allocated block's** payload reserves space for a cached object; the object data is not stored in this simulation.

This is the fundamental shift behind `pool.c`. The rest of this tutorial explains the specific data structures and operations that make it work.

3. The Block Header in This Project

Every block in the pool starts with a `block_header_t`:

```
typedef struct block_header {
    size_t             size;   /* payload capacity (excludes header) */
    int                free;   /* 1 = free, 0 = allocated            */
    struct block_header *next; /* next block header by address        */
} block_header_t;
```

Two constants derived from this struct:

CONSTANT	DEFINITION	VALUE (64-BIT)
`HEADER_SIZE`	`sizeof(block_header_t)`	24 bytes
`MIN_BLOCK_SIZE`	`HEADER_SIZE + 1`	25 bytes

`size` records the payload capacity—the number of usable bytes *after* the header. A block's total footprint in the pool is `HEADER_SIZE + size`. `MIN_BLOCK_SIZE` is the smallest block that makes sense: a header plus at least one payload byte.

Initial pool state after `pool_init`

When the pool is created with, say, 256 bytes, `pool_init` places one header at byte 0. That single block owns the entire pool:

```
block_header_t *first = (block_header_t *)pool->bytes;
first->size = size - HEADER_SIZE;   /* 256 - 24 = 232 */
first->free = 1;
first->next = NULL;
pool->blocks = first;
```

```
  pool->bytes                                              pool end
   |                                                          |
   v                                                          v
  +------------------------+-----------------------------------+
  | header (24 bytes)      | free payload (232 bytes)          |
  | size=232  free=1       |                                   |
  | next=NULL              |                                   |
```

```
+------------------------+--------------------------------------+
byte 0                   byte 24                                byte 256
```

4. Pointer Arithmetic Between Headers and Payloads

Three helper functions let you move between a header and its payload. You will use these constantly—they are provided, not part of your exercise:

`payload_from_header`

Given a header pointer, return the start of the payload by advancing past the header:

```
static inline void *payload_from_header(block_header_t *h) {
    return (void *)((unsigned char *)h + HEADER_SIZE);
}
```

`header_from_payload`

Given a payload pointer, back up to the header that precedes it:

```
static inline block_header_t *header_from_payload(void *payload) {
    return (block_header_t *)((unsigned char *)payload - HEADER_SIZE);
}
```

`pool_offset`

Compute the byte offset of a payload within the pool, used for event output:

```
size_t pool_offset(memory_pool_t *pool, void *payload) {
    return (size_t)((unsigned char *)payload - pool->bytes);
}
```

How it looks

```
  header_from_payload(p) returns here
   |                   payload_from_header(h) returns here
   |                    |
   v                    v
  +--------------------+---------------------------------------+
  | block_header_t     | payload bytes ...                     |
  | (24 bytes)         |                                       |
```

```
+----------------------+--------------------------------------+
byte 0                 byte 24

pool_offset(pool, p) = p - pool->bytes = 24
```

5. Splitting a Block

When the allocator finds a free block larger than the request, it may *split* that block: the front portion becomes the allocated block, and the leftover becomes a new, smaller free block.

The geometry

Suppose the pool has one free block with 232 bytes of payload, and the caller requests 100 bytes. The allocator first rounds the request up to the next multiple of `HEADER_ALIGNMENT` (8 bytes):

```
static inline size_t round_up(size_t val, size_t align) {
    return (val + align - 1) & ~(align - 1);
}
/* round_up(100, 8) = 104 */
```

The new header for the remainder block is placed at `payload + 104`. Before that header is written, `232 - 104 = 128` bytes remain. Since `128 ≥ MIN_BLOCK_SIZE` (25 bytes for a header and at least one payload byte), the split proceeds. After the 24-byte header, the new block has `128 - 24 = 104` payload bytes:

```
  BEFORE (one free block, 232 payload bytes):

  +--------+-------------------------------------------------------+
  | header | free payload: 232 bytes                               |
  | size=232  free=1  next=NULL                                    |
  +--------+-------------------------------------------------------+
  byte 0   byte 24                                          byte 256


  AFTER pool_try_alloc(pool, 100):

  +--------+------------+--------+--------------------------------+
  | header | allocated  | header | free payload: 104 bytes        |
  | size=104  free=0    | size=104  free=1                        |
  | next ----+---------->| next=NULL                               |
  +--------+------------+--------+--------------------------------+
  byte 0   byte 24  byte 128  byte 152                      byte 256
           |             |
          payload        remainder payload
          (104 bytes,    (104 bytes)
           requested 100)
```

The allocated block's payload is 104 bytes (the rounded-up value). The caller asked for 100; the extra 4 bytes are absorbed by alignment. The remainder block starts with its own 24-byte header at byte 128.

> **WHEN NOT TO SPLIT**
>
> If the remainder would be smaller than `MIN_BLOCK_SIZE` (25 bytes), the block is not split. The entire payload stays with the allocated block, which means the block may have more capacity than the caller requested. This avoids creating a uselessly small fragment.

This is student work. The tutorial shows the geometry; you write the code in `split_if_possible`.

6. Coalescing After a Free

When a block is freed, adjacent free blocks should be merged into a single larger free block. This prevents fragmentation from permanently reducing the largest available region.

The steps

1. **Mark the block free:** set `block->free = 1`.
2. **Forward coalesce:** if `block->next` exists and is free, absorb it. Add its size plus `HEADER_SIZE` to yours, and take its `next` pointer.
3. **Backward coalesce:** walk from `pool->blocks` to find the predecessor. If it is free, it absorbs the current block using the same arithmetic.

Example: three blocks, middle one freed

```
BEFORE: block B is freed, neighbors A and C are already free.

+--------+---------+--------+---------+--------+---------+
| hdr A  | free    | hdr B  | FREE    | hdr C  | free    |
| size=80  free=1  | size=64  free=1  | size=40  free=1  |
| next ---+------->| next ---+------->| next=NULL         |
+--------+---------+--------+---------+--------+---------+


STEP 1 — Forward coalesce (B absorbs C):

+--------+---------+--------+--------------------------+
| hdr A  | free    | hdr B  | free: 64 + 24 + 40 = 128  |
| size=80  free=1  | size=128  free=1                    |
| next ---+------->| next=NULL                           |
+--------+---------+--------+--------------------------+


STEP 2 — Backward coalesce (A absorbs B):

+--------+-----------------------------------------------------+
| hdr A  | free: 80 + 24 + 128 = 232                           |
| size=232  free=1                                             |
| next=NULL                                                    |
+--------+-----------------------------------------------------+
```

Each merge emits a `COALESCE` event via `output_event`, reporting the address and new size of the surviving block.

> **FINDING THE PREDECESSOR**
>
> The block list is singly linked, so backward coalescing requires a walk from `pool->blocks` to find the block whose `next` points to the one you just freed. There is no `prev` pointer.

This is student work. The tutorial shows what merging looks like; you write the traversal and pointer updates in `pool_free`.

7. The Allocator Entry Points

The pool exposes two main operations. You know the algorithms from the textbook; here is how they map to the function signatures in this project.

`pool_try_alloc(pool, requested)`

Search the block list for a free block with `size ≥ requested` using the pool's configured policy. If a block is found, split it if possible, mark it allocated (`free = 0`), and return a pointer to its payload. Return `NULL` if no block is large enough.

`pool_free(pool, payload, ev)`

Given a payload pointer previously returned by `pool_try_alloc`, recover the header with `header_from_payload`, mark the block free, then forward-coalesce and backward-coalesce as described in Part 6.

Placement policies

POLICY	SELECTION RULE
`first_fit`	Return the **first** free block with `size ≥ requested`, walking in address order.
`best_fit`	Return the **smallest** free block with `size ≥ requested`, examining every block.

The eviction layer (`cache.c`) calls these functions. When `pool_try_alloc` returns `NULL`, the cache evicts an object to free space and retries. That logic is outside the scope of this tutorial.

> **SUMMARY OF WHAT YOU IMPLEMENT**
>
> Your code goes in `pool.c`. You write the bodies of `first_fit`, `best_fit`, `split_if_possible`, `pool_try_alloc`, and `pool_free`. The helpers (`header_from_payload`, `payload_from_header`, `pool_offset`, `round_up`) and the initialization code (`pool_init`) are provided.

Project 2 materials

The **Project 2 Overview**, **Project 2 Prompt**, and **Pool Tutorial: From Linked Lists to Embedded Free Lists** are all available in **Modules in Canvas**.